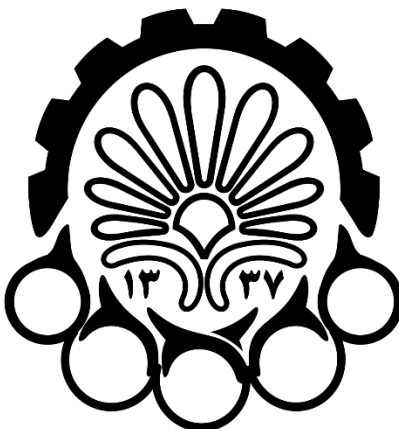


بسمه تعالی



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

موضوع :

گزارش تمرین سوم درس یادگیری ژرف

دانشجو : متین قاسمی

شماره دانشجویی : 402112098

استاد : دکتر مصطفی شمس - دکتر زاهد رحمتی

بهار ۱۴۰۳

سوال 1-a :

در این بخش خواسته شده بود که توابع فعال ساز تعریف شوند که با استفاده از numpy این اتفاق رخ داده است.

سوال 1-b :

برای تعریف کلاس نورون و در متود init ابتدا وزن و بایاس نورون تعریف شده است همان طور که سوال خواسته هم به صورت رندوم و با استفاده از seed داده شده این اتفاق رخ میدهد و هم به صورت وزن دهی اولیه میتوان نورون را تعریف کرد.

```
# Initialize weights and bias for the neuron
np.random.seed(seed)
if input_w is None:
    self.weights = 0.1 * np.random.randn(n_inputs)
    self.bias = 0
else:
    self.weights = np.array(input_w[1:], dtype=np.float64)
    self.bias = input_w[0]
```

در بخش بعدی توابع فعال ساز ممکن به همراه مشتق های آن ها تعریف شده است و در نهایت چک میشود که تابع به صورتی تعریف شده باشد.

```
# Supported activation functions and their derivatives
activation_functions = {
    'relu' : (relu, lambda x: np.where(x > 0, 1, 0)),
    'sigmoid' : (sigmoid, lambda x: sigmoid(x) * (1 - sigmoid(x))),
    'leaky_relu' : (leaky_relu, lambda x: np.where(x > 0, 1, 0.1)),
    'tanh' : (tanh, lambda x: 1 - np.tanh(x) ** 2),
    'sign' : (sign, lambda x: np.where(x > 0, 1, -1)),
    'linear' : (linear, lambda x: 1)}

try:
    self.activation, self.derivative = activation_functions[activation]
except:
    raise Exception("The entered activation function is not supported.")
```

در متود calculate صرفا کافی است که عبارت $wX + b$ برای نورون محاسبه شود سپس تابع فعال ساز روی آن اجرا شود. همچنین مقدار قبل از تابع فعال ساز و ورودی نورون ذخیره شده اند تا در بخش backpropagate بتوانیم از آن ها استفاده کنیم.

```
# Forward pass in neuron: activation(weight * inputs + bias)
self.inputs = np.array(inputs, dtype=np.float64)
self.output = np.dot(inputs, self.weights) + self.bias
return self.activation(self.output)
```

در بخش بعدی متود `backpropagate` پیاده سازی شده است که `error` لایه بعد را میگیرد و سپس از مشتق تابع فعالساز ضرب میشود تا مقدار دلتا محاسبه شود. در نهایت با استفاده از این مقدار وزن و بایاس آپدیت میشوند و در نهایت با ضرب کردن وزن ها در `error` لایه بعد تاثیر نورون در لایه قبلی محاسبه میشود .

```
# Calculate delta based on the error of the previous layer
delta = error * self.derivative(self.output)

# Calculate error of this layer
error = error * self.weights

# Update weights and bias base on delta
self.weights += learning_rate * delta * self.inputs
self.bias += learning_rate * delta

return error
```

سوال 1-c :

هدف این بخش پیاده سازی لایه شبکه عصبی است که به کمک نورون های سوال قبل پیاده سازی میشود. در بخش `init` صرفا کافی است که نورون های مورد نظر را بسازیم و در یک لیست نگه داریم.

```
# Initialize neurons of the layer
self.neurons = []
if input_w is not None:
    for i in range(n_neurons):
        self.neurons.append(Neuron(n_inputs, input_w[i], activation, seed))
else:
    for i in range(n_neurons):
        self.neurons.append(Neuron(n_inputs, None, activation, seed))
```

همچنین در متود `calculate` کافی است خروجی تک تک نورون های لایه را داشته باشیم و در یک لیست ذخیره کنیم.

```
# Forward pass in layer
outputs = []
for neuron in self.neurons:
    output = neuron.calculate(inputs)
    outputs.append(output)
return np.array(outputs)
```

در متود `backpropagate` نیز کافی است `error` هر نورون را بدیم تا هم وزن های نورون آپدیت شود هم در نهایت تاثیر نورون در لایه قبلی هم محاسبه شود. سپس با جمع زدن تاثیر نورون ها میتوانیم میزان `error` لایه قبلی را داشته باشیم.

```

# Calculate the error of each neuron and then sum them to calculate the error of the layer.
layer_errors = np.zeros_like(self.neurons[0].inputs, dtype=np.float64)
for i, neuron in enumerate(self.neurons):
    layer_errors += neuron.backpropagate(learning_rate, errors[i])
return layer_errors

```

سوال 1-d :

برای تعریف شبکه عصبی کافی لایه های آن را مشخص کنیم. بنابراین در متود init ابتدا لایه هارو تعریف میکنیم. همچنین شرایط درست بودن وزن ها و توابع فعال ساز چک شده است.

```

if len(list_of_n_neurons) < 2:
    raise Exception("Cannot make a Neural network with 1 or less layers.")

# Check for valid input regarding activation functions and weights
if (list_of_activation_functions != None and (len(list_of_activation_functions) != len(list_of_n_neurons) or list_of_activation_functions[0] != None))
    raise Exception("List_of_activation_functions is wrong.")
if (list_of_weights != None and (len(list_of_weights) != len(list_of_n_neurons) or list_of_weights[0] != None)):
    raise Exception("List_of_weights is wrong.")

for i in range(1, len(list_of_n_neurons)):
    # Check for weight compatibility if weights are provided
    if list_of_weights != None:
        for item in list_of_weights[i]:
            if len(item) != (list_of_n_neurons[i-1] + 1):
                raise Exception("List_of_weights is entered wrong")

# Create Layers based on input configuration
if list_of_activation_functions == None and list_of_weights == None:
    for i in range(1, len(list_of_n_neurons)):
        self.layers.append(
            Layer(list_of_n_neurons[i], list_of_n_neurons[i - 1], seed=seed))
elif list_of_activation_functions == None and list_of_weights != None:
    for i in range(1, len(list_of_n_neurons)):
        self.layers.append(
            Layer(list_of_n_neurons[i], list_of_n_neurons[i - 1], input_w=list_of_weights[i], seed=seed))
elif list_of_activation_functions != None and list_of_weights == None:
    for i in range(1, len(list_of_n_neurons)):
        self.layers.append(
            Layer(list_of_n_neurons[i], list_of_n_neurons[i - 1], activation=list_of_activation_functions[i], seed=seed))
else:
    for i in range(1, len(list_of_n_neurons)):
        self.layers.append(
            Layer(list_of_n_neurons[i], list_of_n_neurons[i - 1], activation=list_of_activation_functions[i], input_w=list_of_weights[i], seed=seed))

```

در متود feedforward نیز ورودی را درون لایه های شبکه میدهیم و خروجی را میگیریم.

```

# Feedforward through the network's Layers
self.input = inputs
for layer in self.layers:
    inputs = layer.calculate(inputs)
return inputs

```

همچنین برای بخش backpropagate ابتدا یکبار forward رو اجرا میکنیم تا خروجی رو داشته باشیم سپس مشتق تابع خطا رو محاسبه میکنیم. تابع خطای در نظر گرفته شده برای اینجا MSE هستش. پس مشتق خطا به عنوان خطای لایه آخر به شبکه داده میشود و در عملیات backpropagate خطای لایه ها از اخر به اول محاسبه میشود و وزن ها اپدیت میشوند.

```

# Feedforward through the network to calculate outputs
outputs = self.feedforward(self.input)

# Calculate the derivative of the Loss function (MSE) with respect to the outputs
loss_grad = -2 * (outputs - targets)

# Backpropagate through the network's layers
for layer in reversed(self.layers):
    loss_grad = layer.backpropagate(learning_rate, loss_grad)

```

سوال 1-e :

برای دسته بندی با استفاده از شبکه بدون آموزش مدل کافی است خروجی مدل برای نمونه های مختلف را محاسبه کنیم سپس خروجی را رند کنیم که معادل target نمونه باشد . و در نهایت دقت مدل گزارش شده است.

```

def function(input_data, true_labels, network):
    predictions = []
    for input in input_data:
        predictions.append(network.feedforward(input))
    predictions = np.round(predictions).flatten()
    accuracy = ((predictions == true_labels).sum() / len(true_labels)) * 100
    return accuracy

function(input_data, true_labels, network)

```

سوال 1-f :

برای این سوال و برای آموزش مدل کافی است در هر epoch داده ها را به صورت تک به تک به مدل بدهیم و مقدار loss را حساب کنیم و سپس عملیات backpropagate را انجام دهیم تا وزن ها اپدیت شوند. برای هر epoch مقدار loss و دقت را ذخیره میکنیم تا در نهایت در سوال بعد بتوانیم نمایش دهیم. همچنین هر ۵۰ اپیاک مقدار loss و دقت را نشان میدهیم تا مطمئن باشیم مدل به درستی عمل میکند.

```

# Define Learning rate and number of epochs
learning_rate = 0.1
epochs = 1000
n_loss = []
n_accuracy = []

# Training
for epoch in range(1, epochs + 1):
    prediction = []
    loss = []
    for X, y in zip(X_train, y_train):
        # Forward pass
        outputs = network.feedforward(X)
        prediction.append(outputs)
        # Compute Loss (Mean Squared Error)
        loss.append((outputs - y) ** 2)
        # Backpropagate
        network.backpropagate(learning_rate, y)

    # Calculate Loss and accuracy on the training data in each epoch
    prediction = np.round(prediction).flatten()
    accuracy = ((prediction == true_labels).sum() / len(true_labels)) * 100
    n_loss.append(np.mean(loss))
    n_accuracy.append(accuracy)

    if epoch % 50 == 0:
        print(f'Epoch: {epoch} / {epochs}, \t Loss: {np.mean(loss):.3f}, \t Accuracy: {accuracy:.2f}%')

```

در نهایت خروجی چاپ شده از مرحله آموزش مدل به شکل زیر است.

Epoch: 50 / 1000,	Loss: 0.254,	Accuracy: 50.00%
Epoch: 100 / 1000,	Loss: 0.253,	Accuracy: 50.00%
Epoch: 150 / 1000,	Loss: 0.248,	Accuracy: 50.00%
Epoch: 200 / 1000,	Loss: 0.242,	Accuracy: 50.00%
Epoch: 250 / 1000,	Loss: 0.233,	Accuracy: 66.67%
Epoch: 300 / 1000,	Loss: 0.224,	Accuracy: 66.67%
Epoch: 350 / 1000,	Loss: 0.216,	Accuracy: 66.67%
Epoch: 400 / 1000,	Loss: 0.211,	Accuracy: 66.67%
Epoch: 450 / 1000,	Loss: 0.208,	Accuracy: 66.67%
Epoch: 500 / 1000,	Loss: 0.204,	Accuracy: 66.67%
Epoch: 550 / 1000,	Loss: 0.195,	Accuracy: 66.67%
Epoch: 600 / 1000,	Loss: 0.179,	Accuracy: 66.67%
Epoch: 650 / 1000,	Loss: 0.156,	Accuracy: 100.00%
Epoch: 700 / 1000,	Loss: 0.135,	Accuracy: 83.33%
Epoch: 750 / 1000,	Loss: 0.056,	Accuracy: 100.00%
Epoch: 800 / 1000,	Loss: 0.026,	Accuracy: 100.00%
Epoch: 850 / 1000,	Loss: 0.017,	Accuracy: 100.00%
Epoch: 900 / 1000,	Loss: 0.013,	Accuracy: 100.00%
Epoch: 950 / 1000,	Loss: 0.010,	Accuracy: 100.00%
Epoch: 1000 / 1000,	Loss: 0.008,	Accuracy: 100.00%

سوال 1-g :

برای این خواسته شده است که نمودار خطا و دقت مدل را نمایش دهیم که اینکار انجام شده است. برای این منظور از matplotlib استفاده میکنیم.

```
import matplotlib.pyplot as plt
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5))

# Plot Loss vs epochs
ax1.plot(n_loss, label='Loss')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Loss')
ax1.set_title('Loss per Epoch')
ax1.legend()

# Plot accuracy vs epochs
ax2.plot(n_accuracy, label='Accuracy')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Accuracy')
ax2.set_title('Accuracy per Epoch')
ax2.legend()

plt.show()
```

خروجی مقدار loss و accuracy به شکل زیر است. همچنین در این شکل واضح است که مدل درگیر یک نقطه مینیمم محلی شده است و در نهایت از آن عبور کرده است و به دقت ۱۰۰ درصد رسیده است.

